

VMs and Deploying a MERN app

- VMs and Deploying a MERN app
 - Why deploying on the internet, isn't localhost enough ?
 - Domains V/S IPs
 - Localhost
 - Ping command
 - Local Network, routing(mild hosting)
 - Homoglyph attack
 - How to deploy apps (actual hosting) ??
 - What is VM ?
 - Bare metal Servers
 - SSH Protocol, password based auth
 - Key features of SSH :-
 - SSH Protocol, ssh key pair based
 - How to hack without getting noticed and remain for time inside in others laptop ??
 - Generating a new public private keypair
 - Algorithms for public key cryptography

Why deploying on the internet, isn't localhost enough ?

Have you ever seen the below meme ->

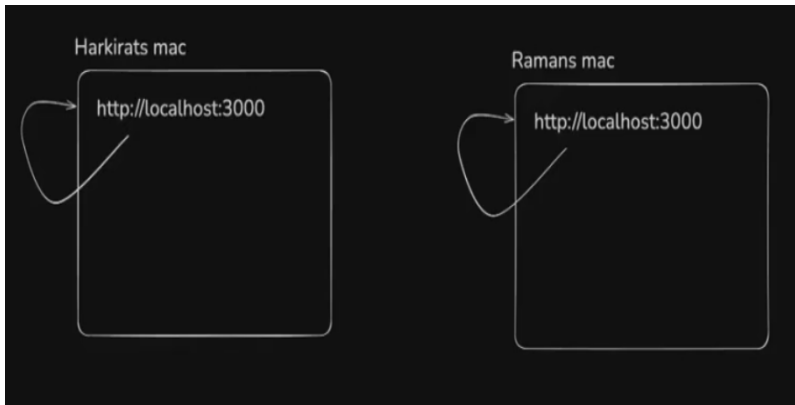


Domains V/S IPs

Localhost

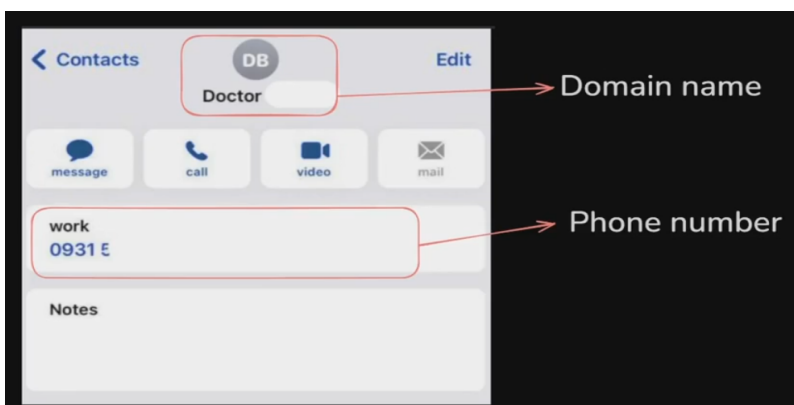
Localhost refers to the computer you're currently working on. **It's essentially a loopback address that points to the machine itself, allowing it to communicate with itself over a network.** In technical terms,

the IP address for localhost is usually **127.0.0.1** for IPv4, or **::1** for IPv6



If you send the address(localhost one) of harkirat to raman, then in the raman machine, it will try to find out **its own localhost**

[!IMPORTANT] > **localhost** is the DOMAIN and **127.0.0.1** is known as IP



IP (Internet Protocol) Address:

- An IP address is a unique string of numbers that identifies a device on a network, like the internet. Think of it like a house address: it's how computers (or any devices on the network) know where to send information.
- For example, **192.168.1.1** is an IP address.
- An IP address is essential for routing data on the internet, but it's not the most human-friendly system.

Domain Name

- A domain name is the readable, human-friendly address we use to access websites, like **google.com** or **facebook.com**

[!NOTE] > **IPv4 V/S IPv6**

IPv4 -> It can range from **0.0.0.0** to **255.255.255.255** (each is 8 bit(0 to 2^8-1 value)) only, Now this lead to the problem that it will be limited which is 256^4 (**only this much IPv4 addresses can be there in the world**) and hence because of this IPv6 was introduced

IPv6 ->

Ping command

Try running

```
ping localhost
```

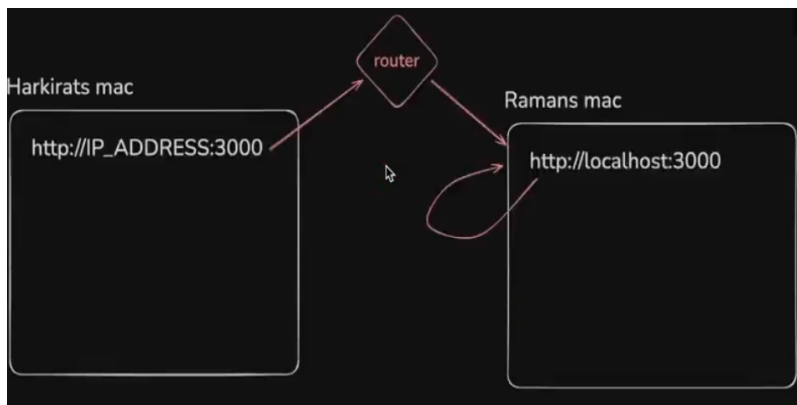
Now try running the below command

```
ping google.com
```

Notice google points to a different IP address

Local Network, routing(mild hosting)

If you have multiple laptops on the same wifi router, you can access one machine from another by using their **private IP address**. This is a mild version of deploying your app on your local network (or **what's called the intranet**)



In the above picture, 2 people are connected to the same network(router) and as **router has a table which is basically storing the ip address so that it can serve it**

and as they both are connected to the same network, so they still be able to see the website on the two device connected without hosting on the internet as you can see above

Steps to follow

Step 1 -> Start a `node.js` process locally on port 3000

```
const express = require('express')
const app = express()

const port = 3000

app.get("/", (req, res) => {
  res.send("Hello World")
})
```

```
app.listen(port, () => {  
  console.log(`Server running on the port ${port}`)  
})
```

Step 2 -> Find the IP of your machine on the local network

```
ifconfig or ipconfig
```

[!IMPORTANT] `ifconfig` or `ipconfig` -> **using this will show you all the Network interfaces (different ways to reach to your computer)**

Finding your wi-fi address by running the above command is simple (just see for the human readable ip address and that is the ip address of the wifi)

You can also use `npx serve` if you are still not able to see the ip address, this gives the ip address of the network your laptop is connected to

also using the above command **you can see all the device contents of the device from which you have ran `npx serve`** in any device which is connected to the same network as that of the device in which `npx serve` is done (try to do it in your windows terminal and go to the given site from any device, you will be able to see your pc contents in any device it returns)

Hosts file

You can override what your domain name resolves to by overriding the hosts file.

```
vi /etc/hosts  
127.0.0.01 harkirat.100xdevs.com
```

Homoglyph attack

Attack in which the hacker makes you visit a website which looks same like the authentic website (UI is copied end to end) hence **tricking the user to enter the sensitive information to fill in the fake website** and hacking the user.

Now you can do the same above thing in your space also. so basically you can point the `facebook.com` ip address to your machine and then whenever someone tries to login from your machine (i.e. connected to your router), then it will go to your machine instead of `facebook.com` and you will be able to get the **username and password** and rest you know

For this ->

Step 1 ->

```
sudo vi /etc/hosts // this has all the list of dns names which you are using or  
you have made for your convenience
```

Step 2 ->

```
// Add a new your ip address for the facebook.com something like the below  
(basically made a new entry in the above file)  
  
127.0.0.1 facebook.com  
// this will make sure that anyone who will search facebook.com and is connected  
to your network will be redirected to 127.0.0.1 (which is your ip address)
```

[!CAUTION] **Make sure to delete the entry from `sudo vi /etc/hosts` time to time nahi to pta chla facebook nhi chal rha(as it is running on local network) and you are blaming the company**

Even if you do

```
ping facebook.com  
  
// Output will be -> 127.0.0.1
```

Basically you have just change the pointing of `facebook.com` to your machine ip address instead of facebook machine

Step 3 -> and now you just have to make a **good signin page of facebook.com such that no one is able to distinguish between the original and yours made** and write the logic to get the contents of the input box of username and password and then re-direct them to original facebook.com so that **unko shak na ho** and also you have not made the page after the login

How to deploy apps (actual hosting) ??

1. Renting servers on a cloud
2. Renting compute yourself in datacenters
3. Self hosting (buying a CPU rack in your house)
4. Serverless providers
5. Cloud native options (k8s)

Great video to look at -> [How to deploy your website](#)

What is VM ?

VM stands for Virtual Machine

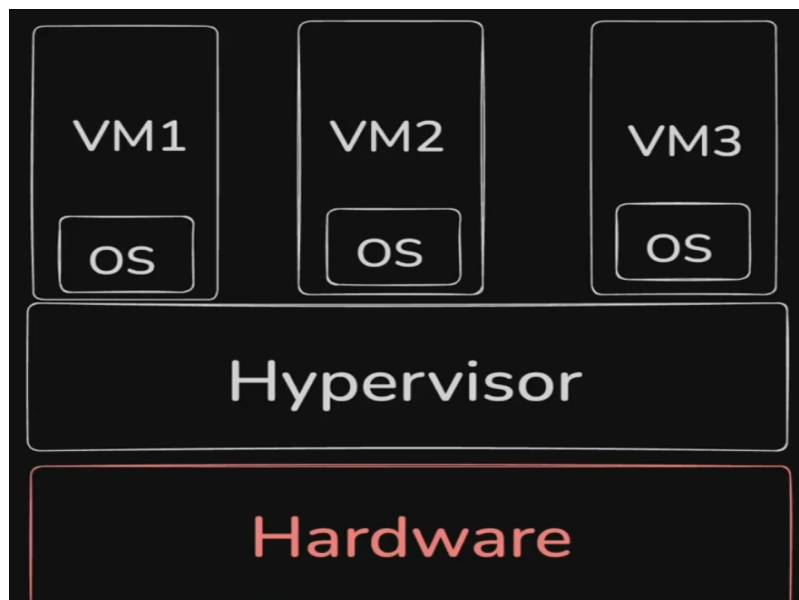


VMS run on a physical server (called the host) but are abstracted through a layer of virtualization software called a **hypervisor** (e.g., VMware, KVM). **This hypervisor divides the host machine's resources (CPU, memory, storage) into separate virtual machines.** (Benefit -> if anyone wants small machine, then it can be provided basically effectively distributing the machine).

Each VM acts like a completely independent machine, even though they share the underlying hardware. You can run different operating systems and applications in different VMS on the same physical server.

VMS are highly flexible and easy to scale. You can quickly spin up, modify, or delete VMs, and you can consolidate multiple workloads on a single server.

The virtualization layer introduces a slight overhead in terms of performance because the hypervisor needs to manage resources and ensure each VM operates independently. However, with modern hypervisors and powerful hardware, this overhead is minimal.



The performance problem present in the Virtual machine is handled by

Bare metal Servers

In a bare-metal setup, **an operating system (OS) runs directly on the physical hardware without a hypervisor in between. There's no virtualization layer.**

Since there's no hypervisor, bare-metal systems tend to offer better performance, as the OS can directly access all the server's resources without sharing them with other instances. This is especially important for high-performance applications like **large databases**, **gaming servers**, or **crypto mining** etc..

But it comes with some drawback too -> With bare-metal, you're typically limited to the resources (CPU, memory, storage) of the actual physical server. **You can't dynamically allocate resources like you can in a VM.** (you can't do like this that if the traffic is high, make all the machine work and when traffic is low, shut down the vm)



SSH Protocol, password based auth

The SSH protocol (Secure Shell) is a **cryptographic network protocol that allows secure communication between two systems**, typically for remote administration. It's most commonly used to log into remote servers and execute commands, but it also facilitates secure file transfers and other operations.

Key features of SSH :-

1. **Encryption:** SSH encrypts the data that's sent between the client and the server, so **even if someone intercepts the connection, they can't read the data**. This makes it much more secure than older protocols like Telnet or FTP, which transmit data in plaintext.

You get a shell or terminal like interface to interact with your machine hosted somewhere

2. **Authentication:** SSH can use two methods of authentication: (below are the only ways to connect to your VM hosted somewhere)
 - **Password-based:** You enter a password to authenticate yourself to the remote system.
 - **Public Key-based:** A more secure method, where the **client uses a private key to authenticate, and the server checks it against the corresponding public key**. This eliminates the need for passwords and provides an extra layer of security.
1. **Integrity:** SSH ensures the integrity of data, meaning that **data cannot be tampered with while it's in transit**. If someone tries to alter the data being sent, the connection will be immediately disrupted.

Till now, we have studied HTTP protocol and this is another protocol

This is important to learn because the most safe way to interact with your VM present on the server.

Now from now onwards you need to have a cloud account on any of the below cloud provider to proceed as this will give you practical knowledge :-

AWS Azure GCP -> Slightly expensive but more reliable

Civo Vulter DigitalOcean -> Economical and cheaper but less reliable

- but for learning purpose, DigitalOcean and Civo are far better and economical for student

Now once you have got a VM and its public IP address, now just go to your terminal (local) and run

```
ssh root@public_ip_you_got
```

after entering the password, you will now enter into the machine but in the machine that is now in the datacenter so **whatever terminal command you will enter here that will get reflected on the machine present on the datacenter** so now if you clone some repo of your project from github and then run **npm run dev**, the website will start to run and you can give the ip to anyone to see your project across internet if you do **mkdir test** then a folder named as test will be created on the machine you rented.

SSH Protocol, ssh key pair based

Before understanding what ssh key pair is, let's first understand why this was needed ??

-> can you notice the problem with SSH protocol from the above paragraph, you have to share the password **for someone else to make changes in this project and collaborate efficiently** but there are many problems to this approach ->

1. It is very hard to identify who made the mistake if something happens wrong in the VM
2. If someone gave it to someone else and that person does some vulnerable task with your VM then your project might get lost.

For the above reason, **ssh keypair came into existence** as soon as people realised that **passwords are not the best option to connect to the machine**

[!CAUTION] SSH Keys are very important from the safety point, if you understand this then you can have access to any of your friend's device for the lifetime access without even making them notice

You have also seen this **SSH** while cloning a project from github (you have 2 options -> **HTTPS** or **SSH**) so **SSH** one is very helpful if you have **PRIVATE repo and then you want to clone that repo**

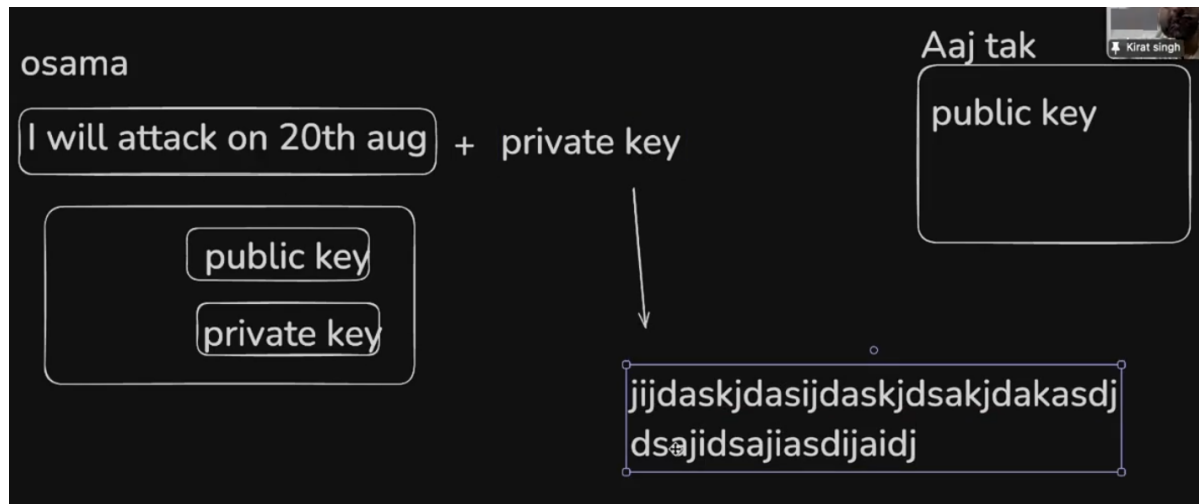
What is public-private keypair ??

-> Before understanding this first understand what is **public key cryptography**

Public key cryptography

-> Lets understand this by taking an example -> Osama bin Laden which was a renowned terrorist and he wanted to make an announcement that he is planning to do something big so to **stay away from CIA, FBI etc.. and convey the message at the same time, he has**

Public - Private key pair as the name suggests he has one **Public key** -> which he gives to everyone (to news agency like Aajtak) and one **Private key** -> which will help him to broadcast the message.



You can see in the above pic that **Public key is available to everyone** and **the message is SIGNED with private key which created a jibish text** now whoever has public key using it they can decode the **Jibish text** and verify also that as it is of Osama's public key so Osama officially has sent this and can find out that he is going to attack and that's how Osama will convey the message without sharing much details about himself.

The above is how the Public-Private keypair and why it is advantageous

In short, the best usecase of it is **you give the public key to everyone** and whenever **you want to convey some message to everyone, just sign it with your private key and send it to everyone** By using the **public key they (everyone) will verify that it is you who have sent this message**

[!IMPORTANT] **Public-Private keypair** are made in such a way that **if you sign a message using private key then it can be decoded only using public key and VICE VERSA is also true**

[!NOTE] **The concept is very similar to JWT authentication (difference between .verify and .decode)** you can see the JSON but can verify only using JWT secret token

How this will solve the above problem ??

-> Lets say now you have a team of developers working on your machine, so now to get access to your machine, they will use their **public key to login** and as soon as they will make some mistake, I can using their **public key** can identify who has made mistake [1st problem solved].

-> also if you want to revoke permissions from some third person, you just need to remove the **public key of that person from your machine and in this way you can control your VM** [2nd problem solved]

Now how this will make you enter into your friend or other machine and remain there for too long is that you just have to put your public key inside your friend's machine (which will basically tell the system that this person can come inside and can access the machine) hence making you enter into your friend's machine.

as long as you have your friends ip and he has ssh port open, i can access whatever i want (even the cookies which might contain some passwords of the renowned site)

Now where these public keys are stored (even you should also check in your laptop) that at below path is there something present if yes, then there are chances that someone has put his/her public key inside your machine and is getting access through it

```
cat ~/.ssh/authorized_keys
```

Run the above command and if you see some file or directory get cautious some one has saved his/her public key here.

How to hack without getting noticed and remain for time inside in others laptop ??

Whenever you get any machine, just run the below command and run this command

```
cat ~/.ssh/authorized_keys
```

Then **Paste you public key here and that's it you are good to go**

Now whenever you want to connect to the other machine just **find out their Ip and run the command**

```
ssh root@the_ip_of_your_friend
```

and as you have already authorised to the others machine by putting yours public key in the other machine so you will get access to the other machine. **BINGO**



How to get their ip address ??

-> Either use the **ping** command in their laptop and see it (too vaigh way to do) or slightly better way is run **npx serve** in your machine and then give link to the other machine now when they will try to see the files using the ip or link you gave them and which this command has provided, you will also get the **ip of the other machine or anyone who has visited your link**



How will you get your public or private key ??

-> Below is the process

Generating a new public private keypair

```
ssh-keygen // ssh ka key generate kr do
```

Running this will ask you where you want to store your public-private key with by-default location to store it (but you can change but make sure to keep it inside the `.ssh` folder only)

Now after this comes the trickier part

Step 2 -> Now you will be asked to put the **Passphrase** which can be anything (even `Enter` will also work), **If this is enabled then you have to enter this passphrase also to get into the other machine**[Its like password to make enter your public key into the other machine]

Step 3 -> Enter or see your public private keypair by running the below command

// you can see the file present by doing `ls` also and then you will see the below files present there

```
cat ~/.ssh/id_rsa.pub // .pub simply means that that is PUBLIC key
cat ~/.ssh/id_rsa // and if not .pub and corresponding same name pair then it
PRIVATE key
```

Step 4 -> Now go to the other's machine and put your public key in that machine in the folder

```
cat ~/.ssh/authorized_keys
```

and now you are good to go **you have hacked your friend's laptop or other machine**

Step 5 -> Just run the below command in your machine whenever you want to get the access of your friend machine

```
ssh root@ip_of_your_friend
```

and you can interact with your friend machines using the terminal.

Algorithms for public key cryptography

The `ssh-keygen` tool can generate SSH key pairs using several different cryptographic algorithms, depending on what you choose during the key creation process. By default, it typically uses RSA, but you can specify other algorithms as well. Here are the most commonly used algorithms:

1. RSA (Rivest—Shamir—Adleman)

- **Default Algorithm** (for most systems): The `ssh-keygen` tool uses RSA by default when creating keys.
- RSA is a widely-used public-key algorithm that provides strong security.

2. Ed25519

- A newer and more secure option: Ed25519 is a modern elliptic curve algorithm that is designed to provide both high security and efficiency.
- It's faster, more secure for the same key size, and less prone to certain vulnerabilities compared to RSA.

```
ssh-keygen -t ed25519
```

3. ECDSA (Elliptic Curve Digital Signature Algorithm)

- Another **elliptic curve algorithm**, which is considered a more secure and efficient alternative to RSA for most use cases.